

Dossier technique — Tri, Statistique, Recherche, Export Excel, ProjectInsertion, Rollback

Ce document décrit en détail le fonctionnement des fonctionnalités suivantes dans le projet "AmineTemplar": tri, statistiques, recherche et export (CSV/excel), et explique en détail `projectinsertion.cpp/.h` et `projectrollback.cpp/.h`.

Table des matières

1. Tri des projets (UI + backend)
 2. Statistiques projet-only (UI + backend)
 3. Recherche (UI + backend)
 4. Export vers CSV/Excel (implémentation, échappement et compatibilité)
 5. `ProjectInsertion` — sauvegarde / chargement des médias
 6. `ProjectRollback` — historique et rollback
 7. Générer un PDF à partir de ce Markdown
-

1) Tri des projets

Code côté backend

- Fichier: `backend/project.cpp`
- Méthodes clés:
 - `QVector<Project> Project::selectAll()` — récupère tous les projets triés par `ID_PROJECT DESC` (SQL order).
 - `QVector<Project> Project::selectAllSortedByTitle(bool ascending)` — tri côté SQL par `UPPER(TITLE)`.
 - `QVector<Project::ProjectOnlyStats> Project::selectProjectOnlyStats(const QString &orderBy = "ID_PROJECT DESC")` — récupère les champs projet (ID, title, description, dates, owner); paramètre `orderBy` permet d'ordonner au niveau SQL.

Notes:

- Le tri peut s'effectuer côté base (ORDER BY), plus scalable pour de grandes tables.

Code côté UI

- Fichier: `ui/projectwidget.cpp`
- Dans la `ProjectStatisticsDialog`, un `QComboBox` (`sortCombo`) propose: `ID (desc)`, `Title (asc)`, `Creation date (desc)`, `Age (desc)`.
- `populate()` appelle `Project::selectProjectOnlyStats()` pour récupérer le dataset, puis applique un `std::sort` en mémoire (client-side) selon la valeur du `sortCombo`.

Pourquoi un tri client-side?

- `Project::selectProjectOnlyStats()` retourne l'ensemble des données, le tri côté client permet des re-sort rapides sans requêter la DB mais n'est pas optimal pour des jeux très larges. Pour une solution basée DB, transmettez `orderBy` et relancez la requête.
-

2) Statistiques projet-only

Backend

- `selectProjectOnlyStats()` construit des instances `Project::ProjectOnlyStats` :
 - `projectId`, `title`, `description`, `creationDate`, `modificationDate`, `ownerId`, `ageDays`.
 - `ageDays` est calculé localement via `creationDate.daysTo(QDate::currentDate())`.

UI

- La dialog `ProjectStatisticsDialog` affiche un `QTableWidget` de 7 colonnes: ID, Title, Creation Date, Modification Date, Age (days), Owner ID, Description.
 - Le label `summaryLabel` affiche:
 - nombre total de projets,
 - âge moyen (jours),
 - première et dernière date de création.
 - L'utilisateur peut trier via le combo (client-side) et exporter la table en CSV.
-

3) Recherche

Backend

- Fichier: `backend/project.cpp`
- Méthode: `QVector<Project> Project::search(const QString &keyword)`
 - Si le `keyword` est vide, retourne `selectAll()`.
 - Échappe les quotes simples: `kw.replace("'", "''")` pour éviter des erreurs SQL.
 - Requête: `WHERE UPPER(TITLE) LIKE UPPER('%kw%') OR UPPER(DESCRIPTION) LIKE UPPER('%kw%')`.
 - `ORDER BY ID_PROJECT DESC`.

UI

- `ui/projectwidget.cpp`
- `onSearchProject(const QString &keyword)` déclenché pendant la saisie (`textChanged`).
- `onSearchProjectClicked()` déclenche la recherche lorsque l'utilisateur clique.
- Les résultats sont affichés via `populateProjectTable(projects)`.

Remarques:

- La recherche est case-insensitive grâce à l'utilisation de `UPPER()`.
 - Pour des recherches avancées: ajouter index FULL-TEXT (si DB supporte) ou utiliser colonnes indexées.
-

4) Export vers CSV/Excel

Backend interface

- Méthode: `bool Project::exportToCsv(const QString &filePath, const QVector<Project> &projects)`
- Header écrit:
`ID,Title,Description,ClientId,ServiceId,CreationDate,ModificationDate,OwnerId.`

Détails CSV & Excel

- Pour protéger les " dans le texte, on remplace " par "" selon la syntaxe CSV. Exemple:
 - `safeTitle.replace("\\\"", "\\\"\\\"");`
 - Ensuite on encadre la chaîne par " pour conserver les virgules à l'intérieur des champs: " + `safeTitle` + "".
- `ProjectStatisticsDialog::exportBtn` utilise un flux similaire pour `ProjectOnlyStats`:
 - Écrit un header différent:
`ID,Title,CreationDate,ModificationDate,AgeDays,OwnerId,Description.`
 - Tronque la description à 200 caractères pour éviter des fichiers trop larges et échappe ".

Localisation CSV / Excel

- Excel attend parfois des ; (point-virgule) comme séparateur selon les paramètres régionaux.
- Si Excel n'ouvre pas correctement le CSV: demander à Excel d'importer le fichier en choisissant le séparateur , ou ;.
- Alternative: produire un XLSX via une librairie (libxlsxwriter ou QXlsx si besoin) pour une compatibilité totale.

5) Détails: `ProjectInsertion` — gestion des médias

- Fichiers: `ui/projectinsertion.cpp` / `ui/projectinsertion.h`
- But: fournir un petit utilitaire pour garder une liste de chemins de médias utilisés dans un projet.
- Comportement:
 1. À la construction: `initializeFilePath()` crée un fichier `media_paths.txt` dans `QDir::currentPath()` (répertoire de l'exécutable / projet).
 2. `loadMediaListFromFile()`:
 - Ouvre le fichier en lecture; lit chaque ligne, `trimmed()` pour retirer les espaces; ignore les lignes vides; retourne un `QStringList`.
 3. `saveMediaListToFile(const QStringList &mediaPaths)`:
 - Ouvre le fichier en écriture et écrit chaque chemin sur une ligne.
 - Si la liste est vide, supprime le fichier pour « garder le dossier propre ».
 4. `getMediaListFilePathForProject(qint64 projectId)` renvoie un chemin projet-spécifique `media_paths_project_{ID}.txt`.
- Logs: la classe envoie des `qDebug()` décrivant l'opération pour faciliter le debug.

Bonnes pratiques (améliorations possibles)

- Placer ces fichiers dans `QStandardPaths::AppDataLocation` pour être indépendant du dossier courant.
- Vérifier les droits d'écriture et afficher une boîte d'erreur si impossible.

6) Détails: `ProjectRollback` — historique et restauration

- Fichiers: `ui/projectrollback.cpp` / `ui/projectrollback.h`
- But: enregistrer un historique d'actions (ajouts/updates) et permettre rollback (restauration par suppression de la dernière action enregistrée).
- Fonctionnement:
 1. `initializeFilePath()` place le fichier `rollback_history.txt` au répertoire courant.
 2. `loadRollbackHistoryFromFile()` lit tout le fichier en `QStringList` (même logique que `ProjectInsertion`).
 3. `saveRollbackHistoryToFile(const QStringList &history)` réécrit tout l'historique dans le fichier; supprime le fichier si vide.
 4. `addAction(const QString &action, const QStringList ¤tHistory)` : ajoute une action puis sauvegarde.
 5. `rollbackLastAction(QStringList &history)` : retire et renvoie la dernière action, puis sauvegarde l'état.
- Remarques:
 - La logique est très simple et n'applique aucune logique transactionnelle à la DB; il ne s'agit que d'un journal local.
 - Pour un rollback réel des changements en base: il faut garder une représentation sérialisée de l'opération (ex: rollback = INSERT/DELETE/UPDATE SQL inverses) et exécuter la commande SQL correspondante.

7) Générer un PDF à partir de ce Markdown

Je fournis plusieurs méthodes — choisissez une selon vos outils:

A) Installer Pandoc + wkhtmltopdf (recommandé pour un rendu précis)

1. Installer Pandoc: <https://pandoc.org/installing.html>
2. Installer wkhtmltopdf (optionnel pour meilleur rendu HTML->PDF) ou utiliser `pandoc --pdf-engine=wkhtmltopdf`.
3. Commande PowerShell (exécuter à la racine du projet):

```
# Installer chocolatey (si vous préférez) ou télécharger les binaires
# Exemple (après avoir installé pandoc):
cd "C:\Users\user\Documents\Qt - Test\16 - 11\Kemicha\docs"
```

```
C:\Program\ Files\Pandoc\pandoc.exe project-features.md -o project-features.pdf
# ou utiliser wkhtmltopdf
C:\Program\ Files\Pandoc\pandoc.exe project-features.md --pdf-
engine=wkhtmltopdf -o project-features.pdf
```

Notes: adaptez le chemin `C:\Program Files\Pandoc\pandoc.exe` selon l'installation.

B) VS Code — extension "Markdown PDF"

1. Ouvrez `docs/project-features.md` dans VSCode
2. Installer l'extension: `yzzhang.markdown-all-in-one` ou `markdown-pdf`
3. Ouvrir la palette (Ctrl+Shift+P) → `Markdown PDF: Export (pdf)`

C) PDF via imprimante virtuelle

1. Ouvrez le Markdown dans VSCode et faites "Print" (Imprimer)
2. Choisissez l'imprimante « Microsoft Print to PDF » pour générer un PDF

Annexes : points techniques et suggestions d'améliorations

- Le tri « côté client » peut être remplacé par une requête dynamique SQL si la taille des données augmente.
- L'export CSV pour Excel peut être enrichi par:
 - ajout d'un encodage (UTF-8 BOM) pour éviter problèmes d'affichage d'accents.
 - création d'un fichier `.xlsx` pour compatibilité améliorée.
- `ProjectRollback`: à intégrer avec transactions SQL, ou un système d'invalidation de cache si plusieurs utilisateurs modifient la base.

Si vous voulez que j'ajoute des captures d'écran ou un export XLSX d'exemple, dites-le — je peux ajouter des étapes ou du code (avec QXlsx) pour générer un fichier Excel natif.